Lecture III - Building Reusable Functions

Programming with Python

Dr. Tobias Vlćek
Kühne Logistics University Hamburg - Fall 2026

📋 Checkpoint 1

The quarterly board review. The first 40 minutes are the checkpoint. It starts now.

- **Individual work**: no AI, no neighbors, no chat
- The **link and QR** are handed out in class. Open it and start
- ~6 short tasks: write code, trace code, fix a bug, answer a multiple choice
- It sweeps **Sessions I–II**: variables, types, `round` and f-strings, `if`/`elif`, `for` loops

When you're done

Menu → *Download* → *Download Python code* → upload the `.py` to the "Checkpoint 1" assignment on Moodle, **before the 40 minutes are up**. No retakes: one sitting.

. . .

> ℹ Note
>
> The green ✅ live checks are **provisional**. The final grading runs on my side. And take a breath: everything in it was rehearsed in the labs.

Episode 3: The Copy-Paste Soup

After the board review

Pens down. The checkpoint is behind you. Now the reason we're all here.

. . .

Tobi has been "reusing" code the only way he knows how: he pasted the same receipt block **14 times**, once per menu item. A ten-cent price change yesterday cost him a whole afternoon of hunting down copies.

. . .

Today the code learns to **reuse itself**. We meet the function, and build a first class.

Writing Functions

The copy-paste pain

Tobi totals each receipt line by hand, the same shape, over and over:

```
print(round(2 * 4.50, 2))    # 2 wraps
print(round(1 * 12.00, 2))   # 1 bowl
print(round(5 * 3.20, 2))    # 5 fries
```

```
9.0
12.0
16.0
```

...

Same calculation, retyped every time. Change the rule once and you must fix it everywhere. There is a better way.

`def`: name the calculation once

Write the calculation **once**, give it a name, and call it whenever you need it:

```
def line_total(qty, price):     # def NAME(inputs):
    return round(qty * price, 2)  # indented body, hands a value back

print(line_total(2, 4.50))
```

```
9.0
```

...

`def` starts the definition · the name is `line_total` · the block is **indented** · `return` hands the answer back.

Parameters vs arguments

- **Parameters** are the names in the definition, the function's inputs: `qty`, `price`
- **Arguments** are the actual values you pass when you **call** it: `2`, `4.50`

```
def line_total(qty, price):      # qty and price are PARAMETERS
    return round(qty * price, 2)

print(line_total(3, 3.20))       # 3 and 3.20 are ARGUMENTS
```

```
9.6
```

...

Same function, different arguments, different result. No retyping.

`return`: hand the value back

`return` sends a value **back to whoever called** the function, so you can store it and use it later:

```
def line_total(qty, price):
    return round(qty * price, 2)

subtotal = line_total(3, 3.20)    # catch what came back
print(subtotal)
print(subtotal + 1.50)            # ...and keep using it
```

```
9.6
11.1
```

Predict: print or return?

`label_price` **prints** but has no `return`. What does the **last line** show?

```
def label_price(price):
    print(f"{price:.2f} EUR")

result = label_price(8.50)
print(result)
```

a) `8.50 EUR` b) `Error` c) `None`

. . .

Predict first. Pick a letter, then I reveal the answer.

Answer: `print` shows, `return` hands back

c) `None`: `label_price` prints its line while running, but with no `return` it hands back `None`. That `None` is what lands in `result`. Printing is not returning.

```
def label_price(price):
    print(f"{price:.2f} EUR")

result = label_price(8.50)    # prints while running...
print(result)                 # ...but the value handed back is None
```

```
8.50 EUR
None
```

Default arguments

A parameter can carry a **default**, used when the caller leaves it out:

```
def service_fee(total, rate=0.05):    # rate defaults to 5 %
    return round(total * rate, 2)

print(service_fee(80))          # default rate → 4.0
print(service_fee(80, 0.10))    # override it → 8.0
```

```
4.0
8.0
```

...

Defaults let one function cover the common case and the special case.

Functions can call functions

A function can use another function you already wrote (reuse builds on reuse):

```
def bill(qty, price):
    items = line_total(qty, price)        # reuse line_total
    return round(items + service_fee(items), 2)  # ...and service_fee

print(bill(2, 4.50))
```

```
9.45
```

...

`line_total` and `service_fee` do their jobs; `bill` just orchestrates. That's how small pieces become a program.

Your turn — 10 minutes

Open the exercise (scan the QR or type the link):

beyondsimulations.github.io/Introduction-to-Python/notebooks/ex_03_a/

First **predict** what happens, then run it.

Scope, then a First Class

Local names stay local

A parameter is the function’s **own private copy**. What happens inside stays inside:

```
def bump(n):
    n = n + 10       # changes the function's OWN copy of n
    return n

print(bump(3))       # 13, the returned value
```

```
13
```

. . .

Names created inside a function live in its **scope**. They vanish when the function ends.

Predict: does the global move?

`bump` adds ten to its parameter. We call it, then print `stock`. What does the **last line** print?

```
def bump(n):
    n = n + 10
    return n

stock = 3
bump(stock)
print(stock)
```

a) `3` b) `13` c) `Error`

. . .

Predict first. Pick a letter, then I reveal the answer.

Answer: the outside variable is untouched

a) `3`: `bump` changes only its own copy `n`. We never stored what it returned, so `stock` out here never moves. The function cannot reach out and rewrite your variables.

```
def bump(n):
    n = n + 10
    return n

stock = 3
bump(stock)      # the return value is thrown away
print(stock)     # still 3
```

```
3
```

Why that's a feature, not a limit

- A function can't **secretly** change your variables, no spooky action at a distance
- You can call it a hundred times and trust your data stays put
- To use a result, you **catch the return** (`stock = bump(stock)`). Deliberate, visible

. . .

Isolation is what makes functions **safe to reuse**. That is the whole point of this episode.

A class bundles data with behavior

Sometimes data and the things you do with it belong **together**. A **class** is a blueprint that bundles both:

- **data**: the facts about one thing (a courier's name, their distance)
- **behavior**: what you can compute from it (their fee)

. . .

A **method** is just a function that lives inside a class and can read the object's own data.

`class`, `__init__`, and `self`

```
class Delivery:
    def __init__(self, courier, distance_km):  # runs when you build one
        self.courier = courier           # store data ON the object
        self.distance_km = distance_km
        self.rate_per_km = 1.20          # every delivery carries its rate

    def fee(self):                        # a method: note self
        return round(self.distance_km * self.rate_per_km, 2)
```

. . .

`__init__` sets up a new object · `self` is *this* object · attributes are stored on `self` and read back through `self`. `fee()` multiplies **two** stored attributes.

Build one, call its method

Instantiate the class to make an object, then use its data and its methods:

```
trip = Delivery("Nadia", 4)   # build one, __init__ runs

print(trip.courier)           # read stored data
print(trip.fee())             # call the method
```

```
Nadia
4.8
```

. . .

`trip` is one `Delivery` object. `trip.fee()` computes from *its own* stored distance: data and behavior, traveling together.

One honest slide about classes

- Today you write **one small class** with an `__init__` and **one method**. That's it
- Classes get tricky fast (inheritance, `self` confusion), and that is **not this course**
- We use them only to *bundle* data with behavior, nothing deeper

. . .

> Tip
>
> If the `self` keyword feels odd right now, that's completely normal. Copy the shape from the worked example. The intuition follows the practice.

Your turn — 10 minutes

Open the exercise (scan the QR or type the link):

beyondsimulations.github.io/Introduction-to-Python/notebooks/ex_03_b/

First **predict** what happens, then run it.

To the Lab

After the break: the lab

- Head to the lab notebook: Episode 3 — The Copy-Paste Soup
- You'll write the fee and tip functions, fix a function that forgets to `return`, give a parameter a default, and build the startup's first `Order` class, ending in the **Tip Calculator Championship**
- It runs entirely in your browser: no setup, just click and code

. . .

> ! Important
>
> **Download your `.py` before you leave.** Closing the tab without downloading loses your work, and downloading is exactly how you just handed in the checkpoint.

Wrap-up

Three things to remember

1. A **function** is written once with `def`, takes **parameters**, and hands a value back with `return`: `print` shows, `return` gives back (no `return` → `None`)
2. Names inside a function are **local**: it can't quietly change your variables, which is exactly what makes it safe to reuse
3. A **class** bundles data (on `self`, via `__init__`) with behavior (methods). You'll write just one small one today

. . .

> i Note
>
> **Next time — Episode 4:** the menu outgrows Tobi's seventeen loose variables (`price1`, `price2`, `price_final_FINAL2`), and nobody can find anything. We give the data a shape: lists and dictionaries.

Literature

Books to start with

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Link to free online version
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

. . .

> i Note
>
> Nothing new here, but these are still great books to start with!

. . .

For more, see the literature list of this course.